

LangChain Classic

源码架构分析与设计说明

langchain-classic v1.0.3 (libs/langchain)

项目	langchain-classic (经典版 LangChain)
版本	v1.0.3
语言	Python >= 3.10
核心依赖	langchain-core >= 1.2.19
框架体系	Pydantic v2 + LCEL (Runnable)
设计模式	模板方法 / 策略 / 观察者 / 组合 / 工厂方法

目录

- 一、项目整体架构概览
- 二、核心模块说明
- 三、核心流程图
 - 3.1 Agent 执行主流程
 - 3.2 Chain 执行流程
 - 3.3 RAG 检索增强生成流程
 - 3.4 Memory 记忆工作流程
 - 3.5 Router Chain 路由流程
- 四、核心时序图
 - 4.1 LLMChain 调用时序
 - 4.2 Agent 推理循环时序
 - 4.3 对话检索链时序
- 五、核心类设计说明
 - 5.1 Chain 基类体系
 - 5.2 Agent 智能体体系
 - 5.3 Memory 记忆体系
 - 5.4 Retriever 检索器体系
 - 5.5 OutputParser 输出解析器
 - 5.6 Callback 回调体系
- 六、设计模式总结

一、项目整体架构概览

langchain-classic 是 LangChain 生态的核心编排层，负责将 langchain-core 提供的基础抽象（Runnable、BaseRetriever、BaseChatModel 等）和 langchain-community 提供的第三方集成组合成完整的应用。它是构建 LLM 应用的主要入口点。

架构分层

层次	包名	职责
核心层	langchain-core	基础抽象、接口、Runnable 协议、LCEL 表达式语言
编排层	langchain-classic	Chain、Agent、Memory、Retriever、Evaluation 等具体实现
集成层	langchain-community	170+ 文档加载器、80+ LLM、60+ Embedding、80+ VectorStore
伙伴层	partners/*	OpenAI、Anthropic、Ollama 等官方维护的集成包

核心模块结构

模块	目录	说明	核心类
Agent	agents/	LLM 驱动的决策引擎，支持 8+ 种 Agent 类型	AgentExecutor, Agent, RunnableAgent
Chain	chains/	多组件组合的可复用调用序列	LLMChain, SequentialChain, RetrievalQA
Memory	memory/	维护对话状态和上下文	ConversationBufferMemory, EntityMemory
Retriever	retrievers/	根据查询返回相关文档	MultiQueryRetriever, SelfQueryRetriever
OutputParser	output_parsers/	解析 LLM 输出为结构化数据	StructuredOutputParser, PydanticOutputParser
Callback	callbacks/	监听执行事件的观察者模式	BaseCallbackHandler, CallbackManager
Evaluation	evaluation /	对 LLM 输出进行评分评估	StringEvaluator, PairwiseStringEvaluator

二、核心模块说明

2.1 延迟加载与向后兼容机制

langchain-classic 采用 `__getattr__` 延迟加载模式。主入口 `__init__.py` 中通过 `_api/module` 模块的 `create_importer()` 函数，实现了带废弃警告的动态导入。这使得包的导入时间极短，同时保持了与旧版 API 的兼容性。所有导出的 44 个符号均带有废弃警告，推荐用户迁移到 langchain-core 或 langchain-community 中的对应实现。

2.2 Runnable 与 LCEL 表达式语言

LangChain 表达式语言 (LCEL) 是基于 `Runnable` 协议的组合式编程模型。`Chain` 基类继承自 `RunnableSerializable`，使得所有 `Chain` 都可以通过管道操作符 `|` 进行组合。新版代码全面拥抱 LCEL，如 `create_stuff_documents_chain()` 直接返回 `Runnable` 管道：`RunnablePassthrough.assign(context=format_docs) | prompt | llm | output_parser`。

2.3 工厂方法模式

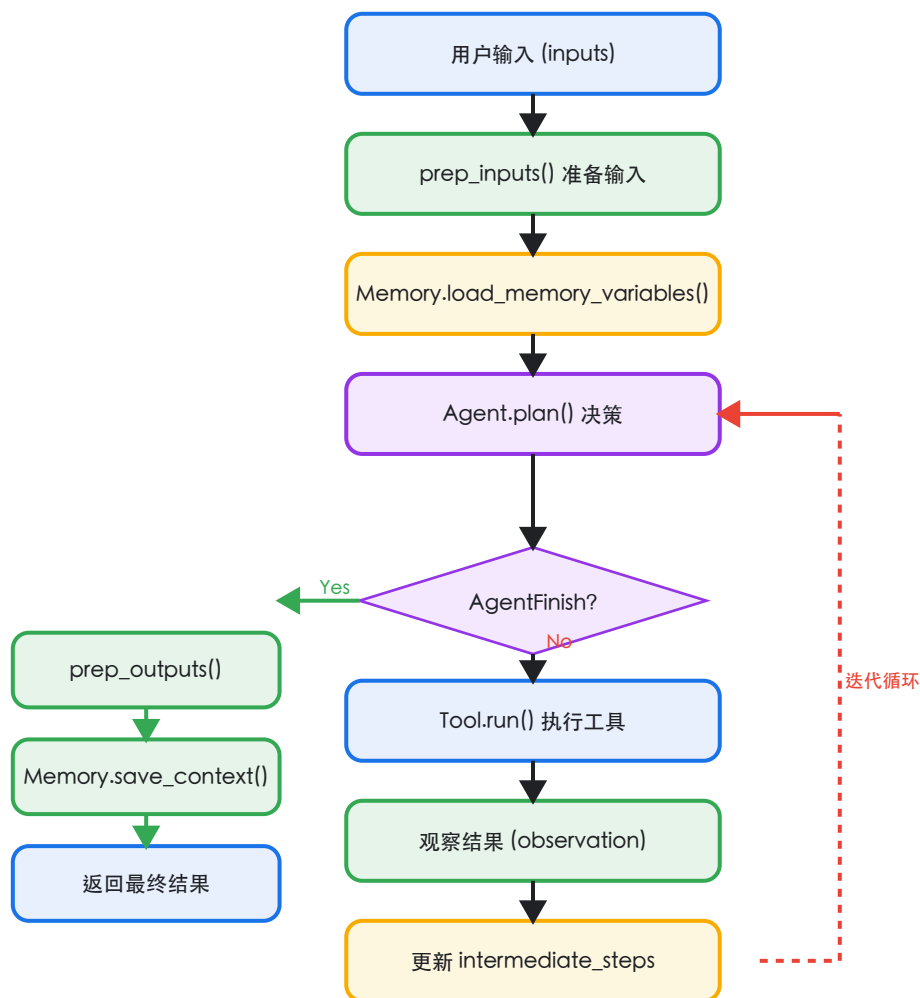
几乎所有核心类都提供了 `from_llm()` 或 `from_llm_and_tools()` 类方法，封装了复杂的构造逻辑。例如 `RetrievalQA.from_chain_type(llm, chain_type='stuff', retriever=...)` 会自动创建 `LLMChain` + `StuffDocumentsChain` + `RetrievalQA` 的完整链路。

三、核心流程图

3.1 Agent 执行主流程

AgentExecutor 是 Agent 的核心执行引擎，它驱动一个"思考-行动-观察"的迭代循环，直到 Agent 返回 AgentFinish 或达到最大迭代次数/最大执行时间。每次迭代中，Agent 通过 plan() 方法调用 LLM 进行决策，然后 AgentExecutor 执行对应的 Tool 并将观察结果追加到 intermediate_steps 中。

Agent 执行主流程图 (AgentExecutor)

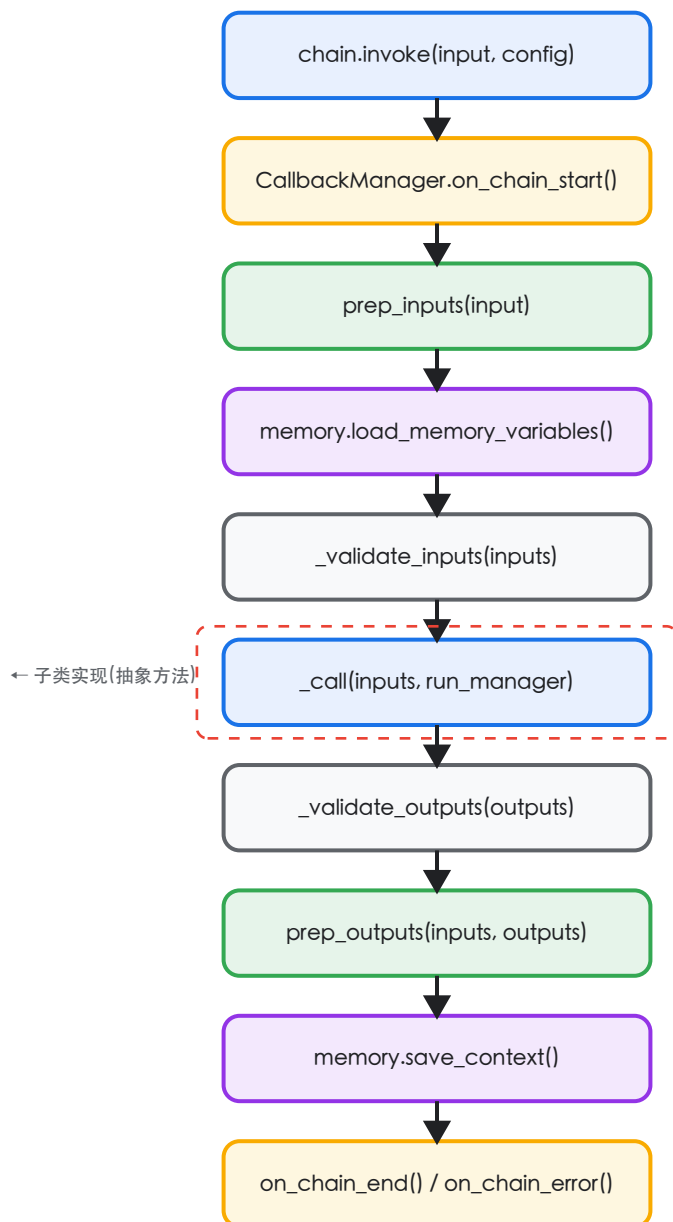


回调事件: on_chain_start → on_llm_start → on_agent_action → on_tool_start → on_tool_end → on_chain_end

3.2 Chain 执行流程 (模板方法模式)

Chain 基类采用经典的模板方法模式。invoke() 方法定义了完整的执行骨架：准备输入 → 加载记忆 → 验证输入 → 执行核心逻辑 → 验证输出 → 保存记忆 → 触发回调。子类只需实现抽象方法 _call()。

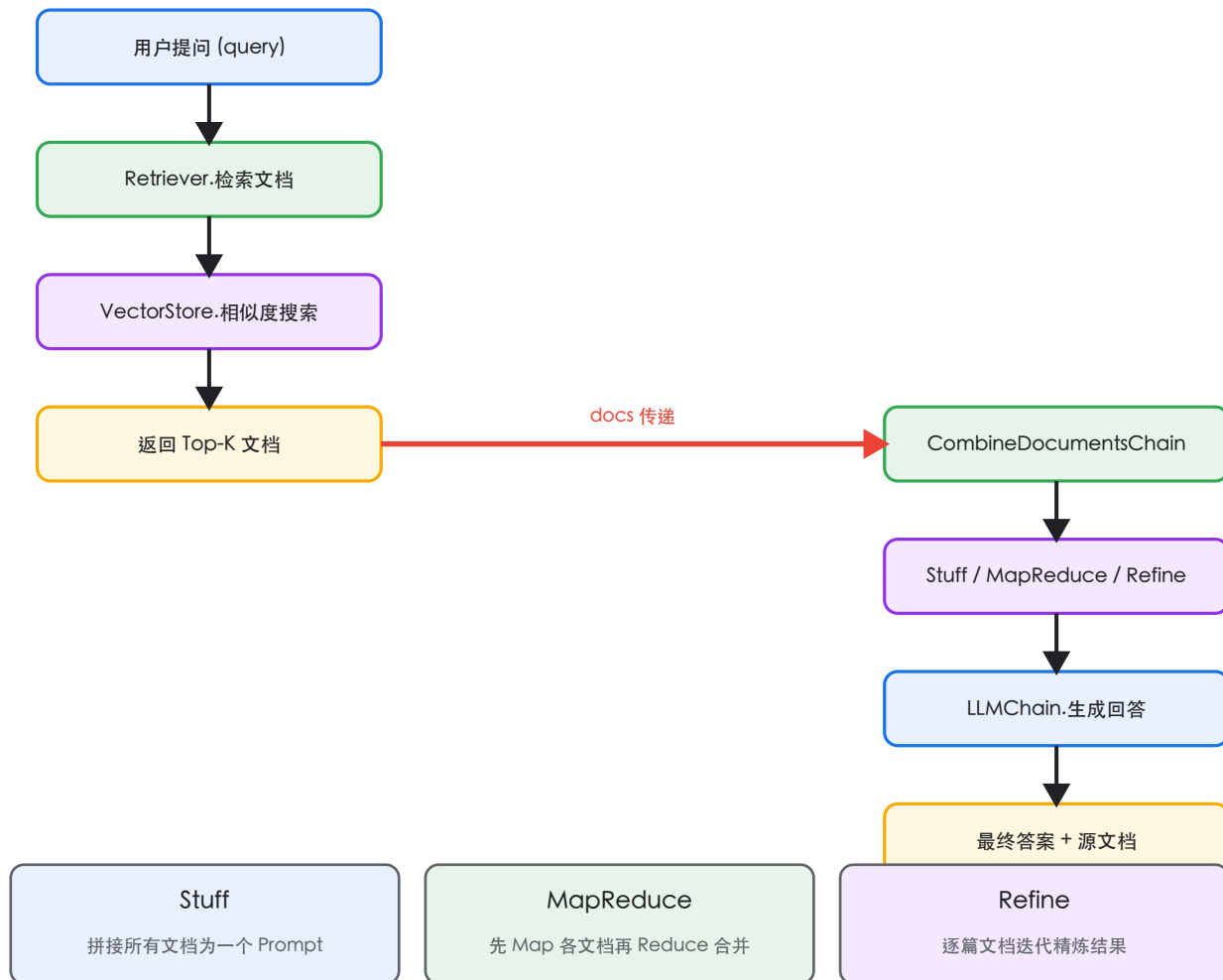
Chain 执行流程图 (模板方法模式)



3.3 RAG 检索增强生成流程

RetrievalQA 实现了经典的 RAG 模式：先通过 Retriever 检索相关文档，再通过 CombineDocumentsChain 将文档与问题组合后交给 LLM 生成回答。支持三种文档合并策略：Stuff（直接拼接）、MapReduce（分治合并）、Refine（迭代精炼）。

RAG 检索增强生成流程图 (RetrievalQA)

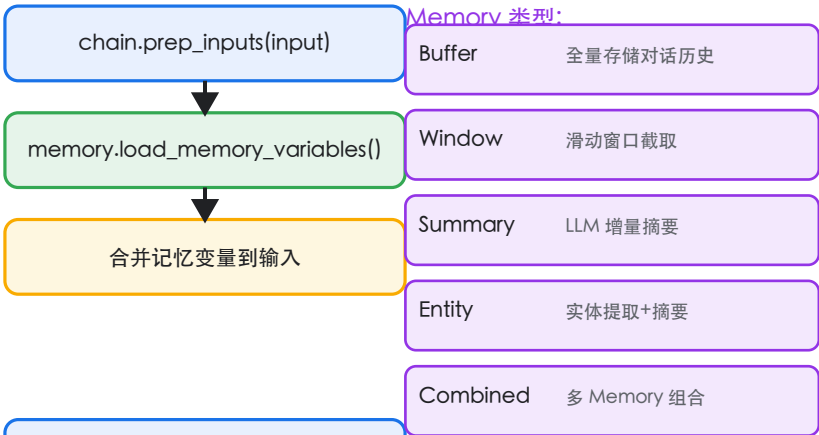


3.4 Memory 记忆工作流程

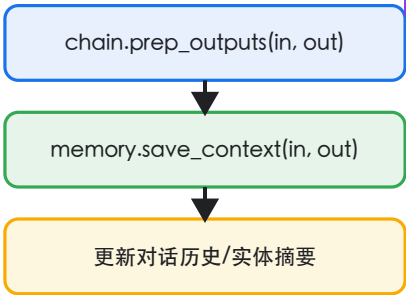
Memory 系统与 Chain 紧密协作：在 Chain 执行前通过 `load_memory_variables()` 加载历史上下文，在执行后通过 `save_context()` 保存新的对话内容。支持多种记忆策略，从简单的全量存储到基于 LLM 的智能摘要。

Memory 记忆系统工作流程

① Chain 执行前（加载记忆）



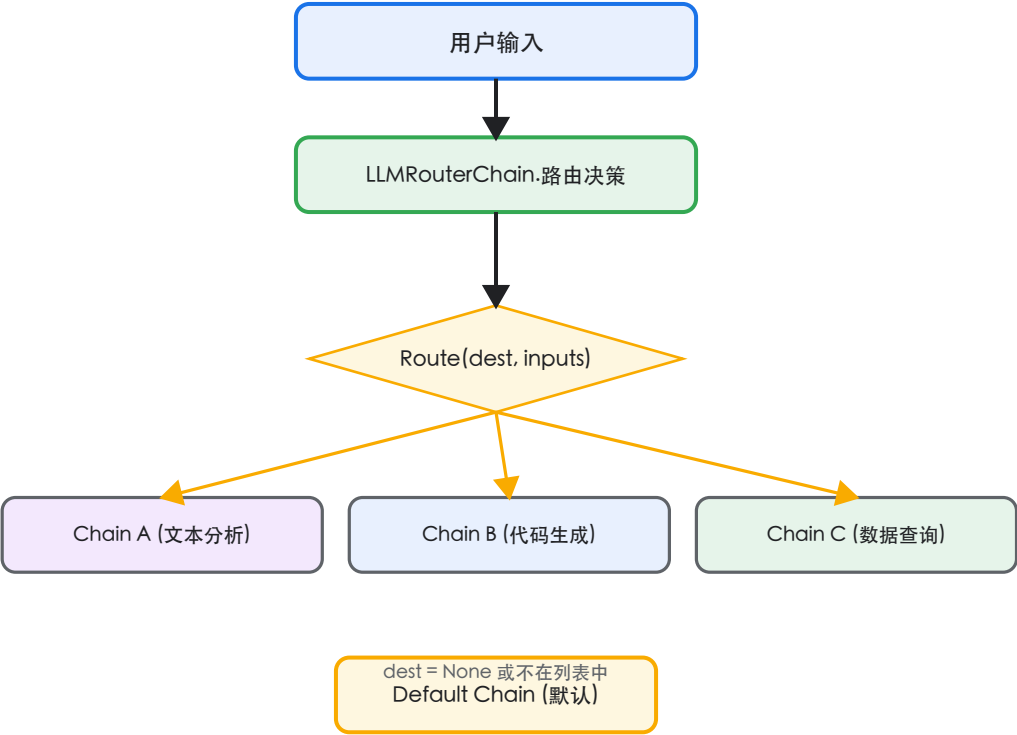
② Chain 执行后（保存记忆）



3.5 Router Chain 路由流程

MultiRouteChain 实现了动态路由分发机制。LLMRouterChain 使用 LLM 分析用户输入，输出目标链名和处理后的输入，MultiRouteChain 根据路由结果分发到对应的目标链执行。

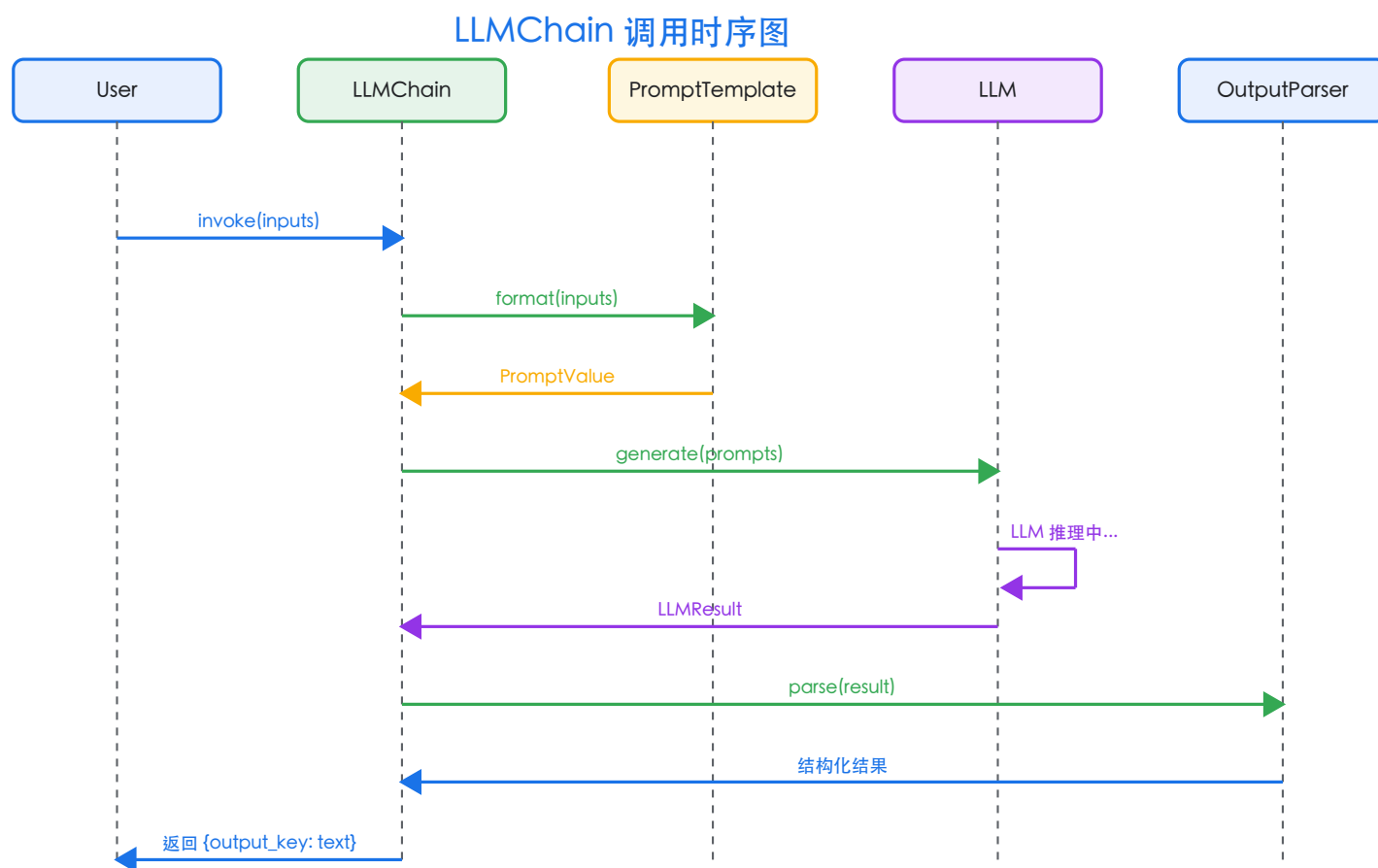
MultiRouteChain 路由流程图



四、核心时序图

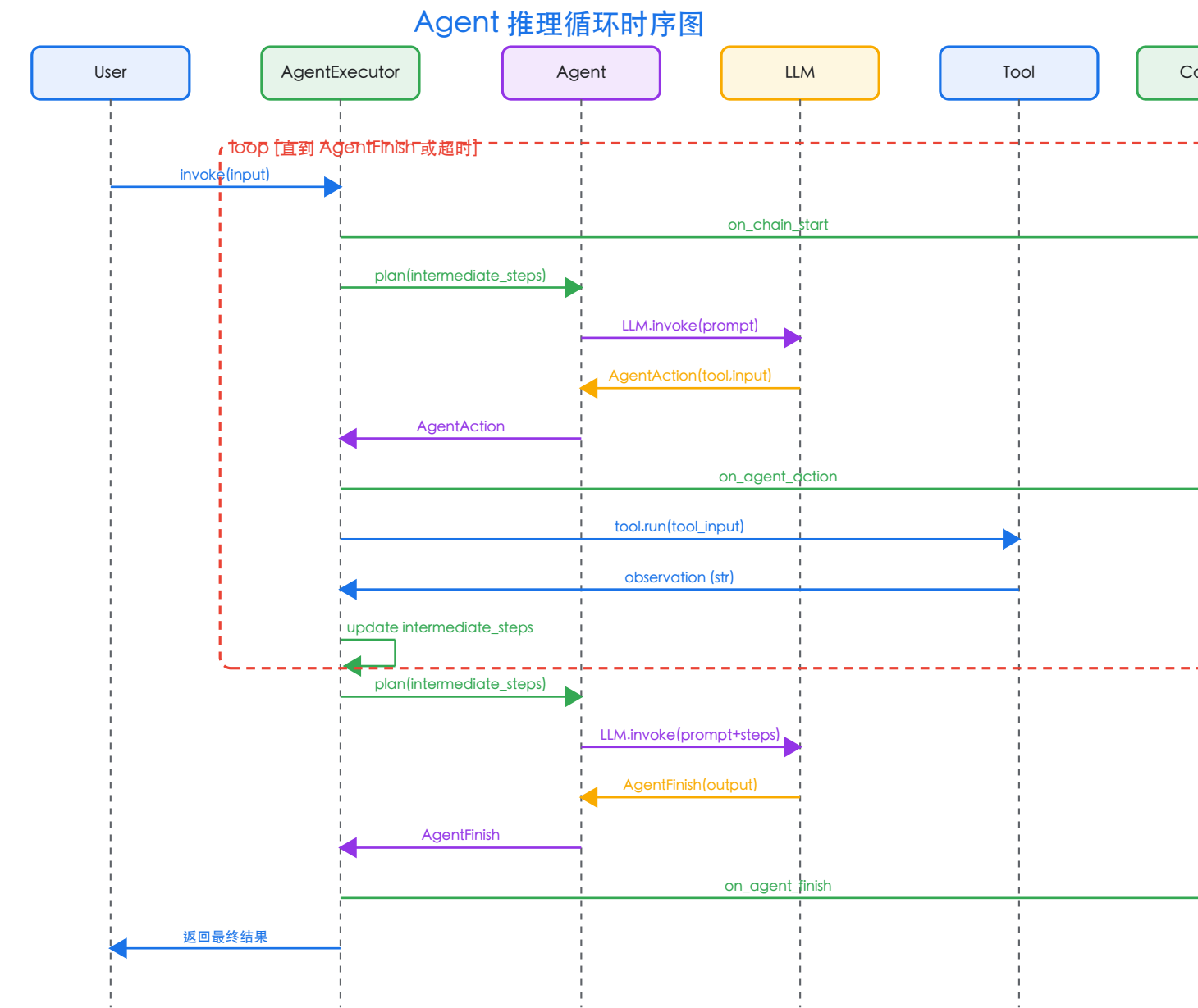
4.1 LLMChain 调用时序图

LLMChain 是最基础的链类型，实现了 Prompt → LLM → OutputParser 的三步管道。它被 Agent、StuffDocumentsChain、ConversationSummaryMemory 等大量组件依赖。新版推荐使用 LCEL 替代：prompt | llm | parser。



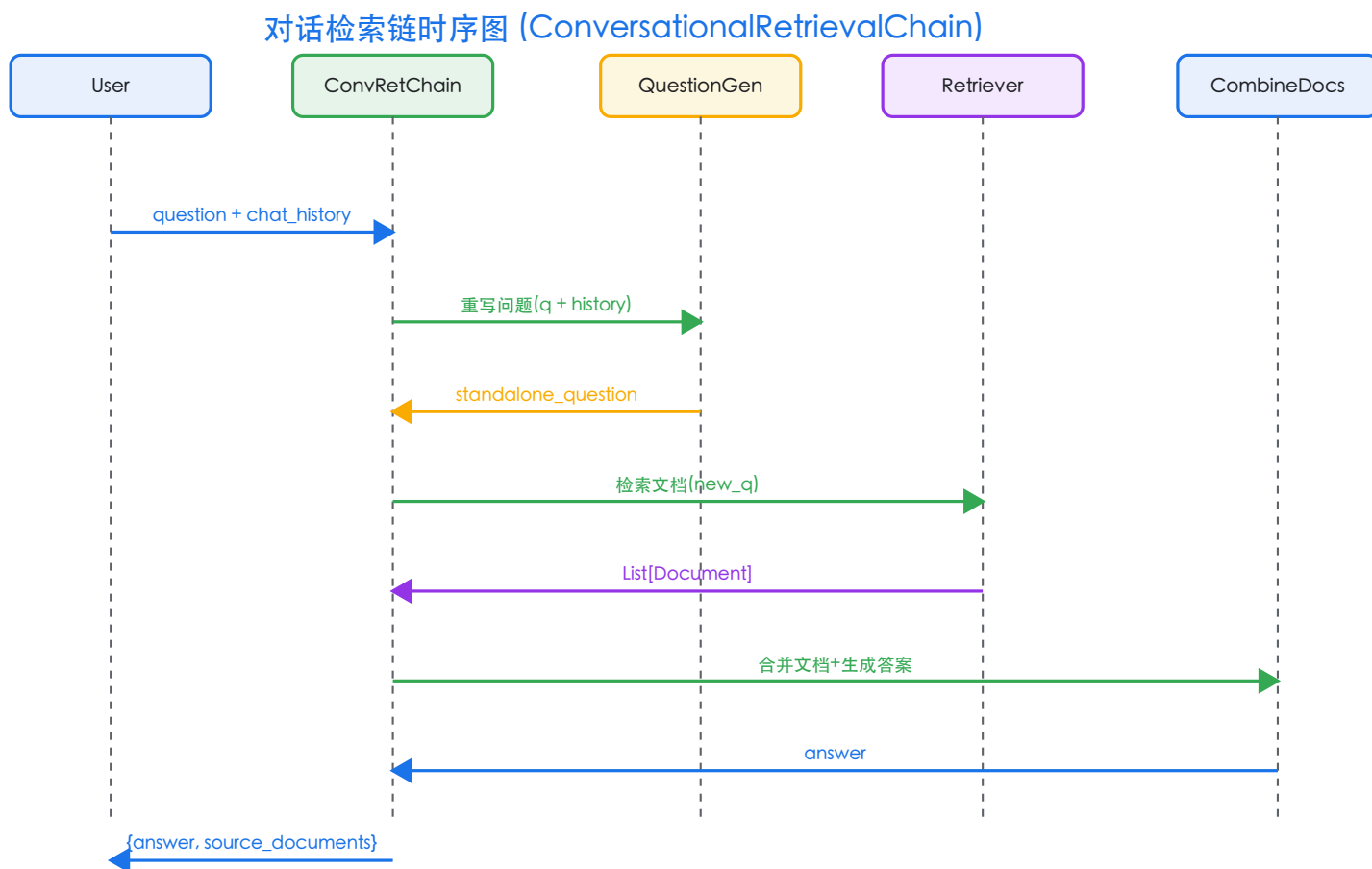
4.2 Agent 推理循环时序

Agent 的推理循环涉及 6 个参与者：User、AgentExecutor、Agent、LLM、Tool 和 Callback。AgentExecutor 协调整个流程，Agent 负责决策使用哪个 Tool，Callback 负责监听和记录事件。循环直到返回 AgentFinish 或达到限制。



4.3 对话检索链时序

ConversationalRetrievalChain 实现了带对话历史的 RAG 流程。它先用 QuestionGenerator 将当前问题和历史对话组合重写为独立问题，然后检索文档，最后通过 CombineDocumentsChain 生成回答。



五、核心类设计说明

5.1 Chain 基类体系

Chain 是所有链的抽象基类，继承自 RunnableSerializable[dict, dict]。它实现了模板方法模式，定义了完整的执行骨架，子类只需实现 _call() 抽象方法。

类名	继承	职责	核心方法
Chain	RunnableSerializable	所有链的抽象基类，定义执行骨架	invoke(), _call(), prep_inputs(), prep_outputs()
LLMChain	Chain	Prompt+LLM+Parser 的基础管道	generate(), prep_prompts(), create_outputs()
SequentialChain	Chain	多个链的串联执行，前序输出作为后序输入	_call() 串联调用子链
RetrievalQA	BaseRetrievalQA	检索+文档合并+生成的RAG链	_get_docs(), from_chain_type()
StuffDocumentsChain	BaseCombineDocsChain	将所有文档拼接为一个Prompt	combine_docs(), _get_inputs()
MapReduceDocsChain	BaseCombineDocsChain	先Map各文档再Reduce合并	combine_docs(): Map→Reduce
RefineDocsChain	BaseCombineDocsChain	逐篇文档迭代精炼结果	combine_docs(): initial→refine×N
ConvRetrievalChain	Chain	带对话历史的检索问答链	question_generator + retriever + combine
MultiRouteChain	Chain	根据LLM路由结果分发到目标链	router_chain.route() → dest_chain

5.2 Agent 智能体体系

Agent 体系分为三层：Agent（决策层）、AgentExecutor（执行层）和 Tool（工具层）。Agent 负责调用 LLM 决定使用哪个 Tool，AgentExecutor 负责协调整个推理-行动-观察循环，Tool 封装了与外界交互的能力。

类名	职责	设计模式
BaseSingleActionAgent	单动作 Agent 抽象基类，定义 plan() 接口	策略模式
BaseMultiActionAgent	多动作 Agent 抽象基类，可同时返回多个 Action	策略模式
RunnableAgent	将 Runnable 包装为 Agent 接口的适配器	适配器模式
Agent (经典)	基于 LLMChain + agent_scratchpad 的经典 Agent	模板方法
AgentExecutor	核心执行引擎，驱动思考-行动-观察循环	策略+迭代器
AgentExecutorIterator	将 Agent 循环暴露为 Python 迭代器	迭代器模式
AgentOutputParser	解析 LLM 输出为 AgentAction/AgentFinish	解释器模式

Agent 类型枚举支持 8 种 Agent : ZERO_SHOT_REACT、REACT_DOCSTORE、SELF_ASK_WITH_SEARCH、CONVERSATIONAL_REACT、CHAT_ZERO_SHOT_REACT、CHAT_CONVERSATIONAL_REACT、STRUCTURED_CHAT_ZERO_SHOT_REACT、OPENAI_FUNCTIONS。

5.3 Memory 记忆体系

Memory 体系负责维护 Chain 的状态。BaseMemory 定义了 load_memory_variables() 和 save_context() 两个核心接口。BaseChatMemory 扩展了聊天消息存储能力。各种具体实现提供了不同的记忆策略。

类名	策略	特点
ConversationBufferMemory	全量存储	存储所有对话消息，简单但可能超出 token 限制
ConversationBufferWindowMemory	滑动窗口	只保留最近 K 轮对话
ConversationSummaryMemory	LLM 摘要	每次对话后用 LLM 增量更新摘要，节省 token
ConversationSummaryBufferMemory	摘要+缓冲	最近消息保留原文，超出部分自动摘要
ConversationEntityMemory	实体提取	用 LLM 提取实体并维护实体摘要（支持多种存储后端）
CombinedMemory	组合	将多个 Memory 的变量合并为统一视图（组合模式）

5.4 Retriever 检索器体系

Retriever 负责根据查询返回相关文档。BaseRetriever 定义了 _get_relevant_documents() 接口。langchain-classic 提供了多种增强型检索器：

类名	策略	设计模式
ContextualCompressionRetriever	先检索再压缩：base_retriever → base_compressor	装饰器模式
MultiQueryRetriever	LLM 生成多个查询变体 → 并行检索 → 去重合并	查询扩展模式
SelfQueryRetriever	LLM 生成结构化查询 → Visitor 翻译 → VectorStore 搜索	访问者模式
MultiVectorRetriever	为文档存储多种嵌入表示	策略模式
ParentDocumentRetriever	检索子文档，返回父文档	组合模式
EnsembleRetriever	多个检索器结果的加权融合	集成模式

5.5 OutputParser 输出解析器

OutputParser 负责将 LLM 的文本输出解析为结构化数据。核心接口是 parse(text) → T。

解析器	输出类型	说明
StructuredOutputParser	dict[str, Any]	基于 ResponseSchema 定义的 JSON 结构解析
PydanticOutputParser	Pydantic Model	自动生成格式指令，解析为 Pydantic 模型
OutputFixingParser	T	解析失败时用 LLM 修复输出

RetryOutputParser	T	解析失败时重试 LLM 调用
AgentOutputParser	AgentAction AgentFinish	将 LLM 输出解析为 Agent 的行动或结束指令

5.6 Callback 回调体系

Callback 体系基于观察者模式，贯穿 LangChain 所有组件的执行过程。BaseCallbackHandler 通过多个 Mixin 组合了所有事件钩子。CallbackManager 负责管理和分发事件通知。

核心事件链：on_chain_start → on_llm_start → on_llm_new_token → on_llm_end → on_agent_action → on_tool_start → on_tool_end → on_agent_finish → on_chain_end。每个组件在执行时都会通过 RunManager 触发对应的回调事件。

六、设计模式总结

langchain-classic 在架构设计中大量运用了经典设计模式，以下是核心模式的总结：

设计模式	应用位置	说明
模板方法	Chain._call(), BaseCombineDocumentsChain.combine_docs()	基类定义执行骨架，子类实现核心逻辑
策略模式	Stuff/Refine/MapReduce; 各 OutputParser; 各 EntityStore	可替换的算法和存储实现
观察者模式	Callback 体系 (BaseCallbackHandler → CallbackManager)	事件通知机制贯穿所有组件的执行
迭代器模式	AgentExecutorIterator.__iter__/__aiter__	将 Agent 推理循环暴露为 Python 迭代器
组合模式	CombinedMemory, SequentialChain, MultiRouteChain	多个同类型组件组合为一个
工厂方法	from_llm() 类方法 (RetrievalQA, SelfQueryRetriever 等)	封装复杂的对象构建逻辑
建造者模式	VectorstoreIndexCreator	分步构建向量索引
访问者模式	SelfQueryRetriever 的 StructuredQueryTranslator	将统一查询翻译为各向量库方言
装饰器模式	ContextualCompressionRetriever	包装基础检索器，叠加压缩处理
适配器模式	RunnableAgent, format_scratchpad 函数族	将 Runnable/内部数据适配为目标接口
门面模式	VectorStoreIndexWrapper	隐藏链式构建的复杂性

总结

langchain-classic 作为 LangChain 生态的核心编排层，通过精心设计的抽象体系和丰富的设计模式，实现了高度的可扩展性和可组合性。其核心架构可以概括为：

- Chain 是骨架：模板方法模式确保了统一的执行流程（输入准备 → 记忆加载 → 核心逻辑 → 记忆保存 → 回调通知）
- Agent 是大脑：策略+迭代器模式驱动了「思考-行动-观察」的智能循环
- Memory 是记忆：多种策略模式支持从简单缓冲到 LLM 智能摘要的记忆管理
- Retriever 是感知：装饰器+访问者模式实现了灵活的文档检索增强
- Callback 是神经：观察者模式贯穿所有组件，提供完整的执行监控能力
- LCEL 是未来：基于 Runnable 协议的组合式编程正在逐步替代经典的 Chain 类